Sentence Memorability Graphs

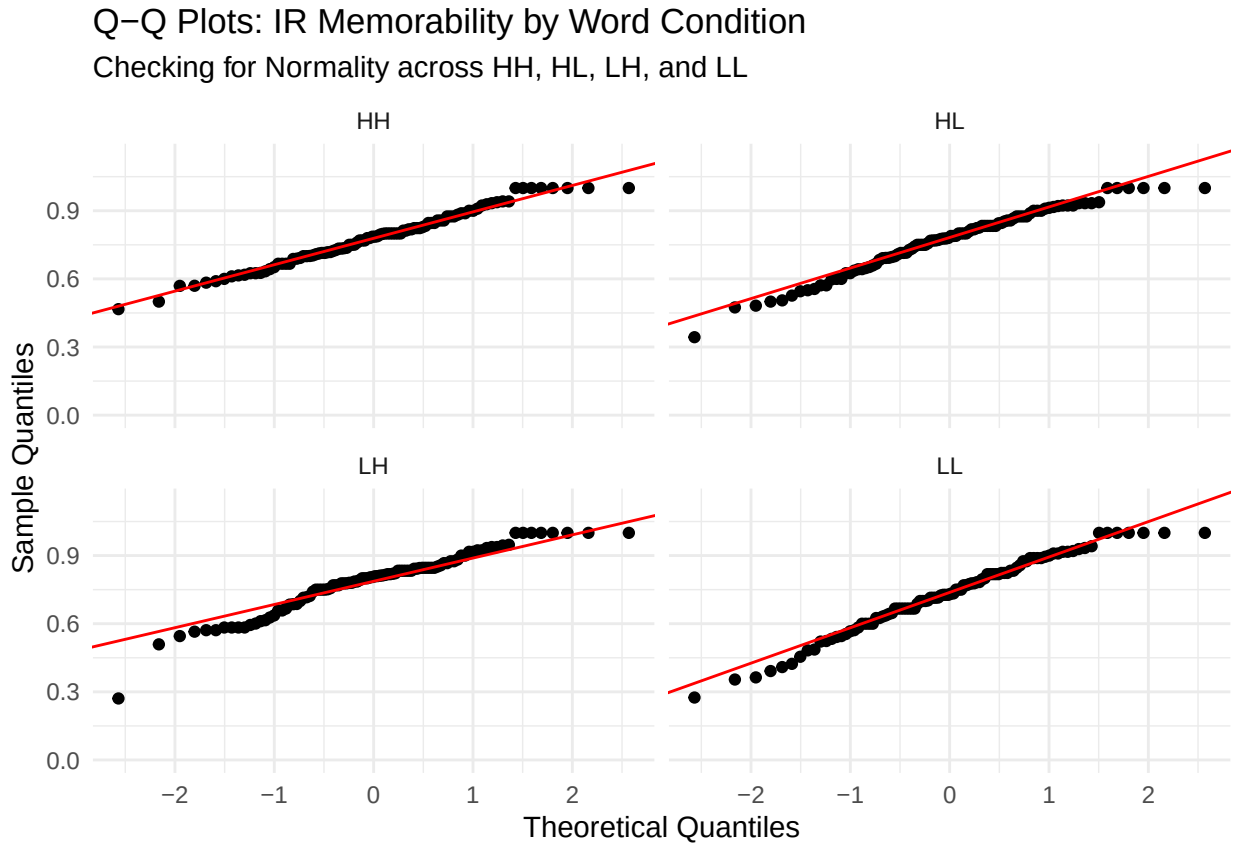
```r
raw_dataset <- read_csv(
  "sentence_memorability_data.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

aggregated_matrix <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  mutate(
    # mapping conditions
    Word_Condition = case_when(
  str_detect(Stimulus, "^HH_")  ~ "HH",
  str_detect(Stimulus, "^HVL_") ~ "HL",
  str_detect(Stimulus, "^LVH_") ~ "LH",
  str_detect(Stimulus, "^LVL_") ~ "LL"
),
    Word_Condition = factor(Word_Condition, levels = c("HH","HL","LH","LL")),

    Sentence_Voice = case_when(
      str_detect(Stimulus, "_A$") ~ "Active",
      str_detect(Stimulus, "_P$") ~ "Passive",
      TRUE ~ NA_character_
    ),
    Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),

    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),

    # store previous IR result
    prev_IR = lag(Accuracy.IR)
  ) %>%
  group_by(Stimulus, Word_Condition, Sentence_Voice) %>%
  summarise(

    # IR components
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown"),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy.IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    # IR metrics
    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,
```

```
    # WR components (after correct IR)
    WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_total   = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

    # WR metric
    WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

    .groups = "drop"
  ) %>%
  mutate(WR_Accuracy = replace_na(WR_Accuracy, 0))
```

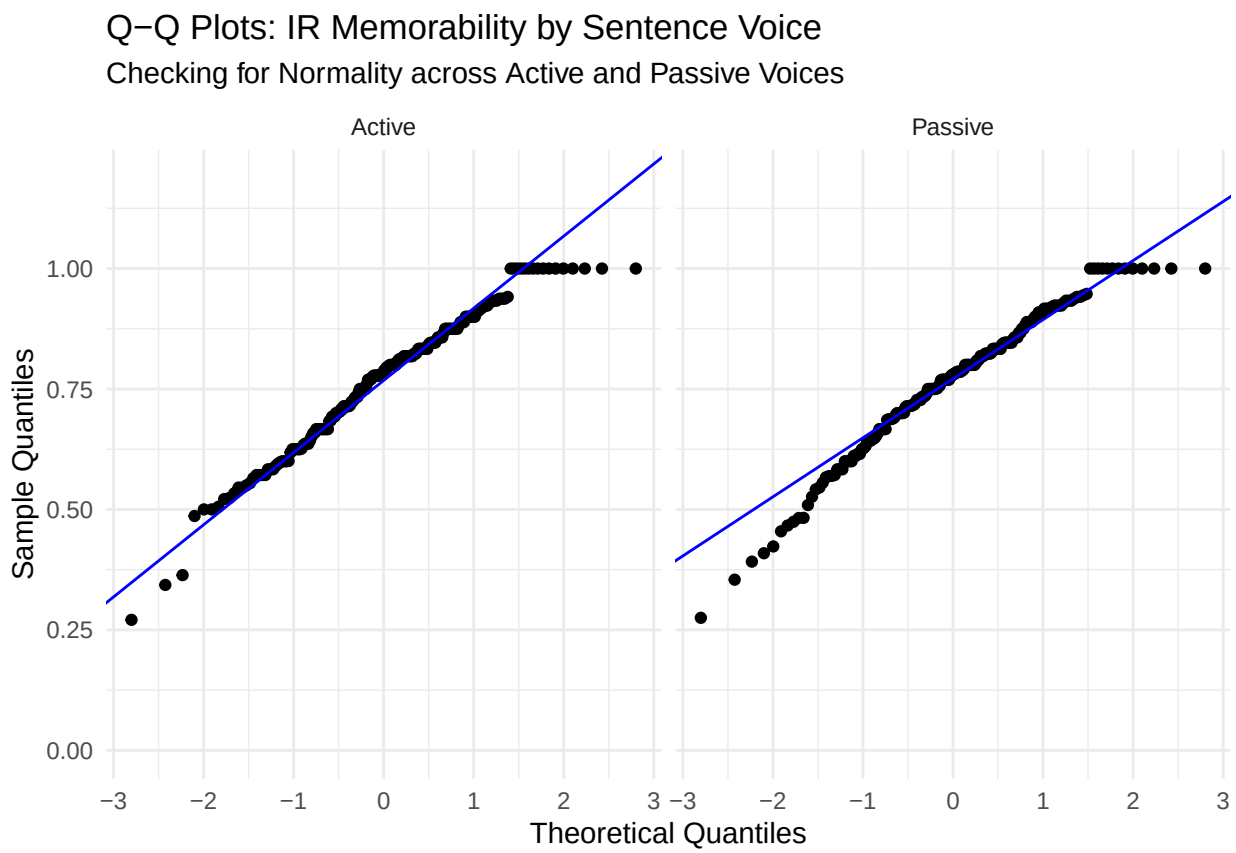
```
# 1. Assign Block Numbers using 5-minute gaps
time_blocked_data <- raw_dataset %>%
  # 1. Sort the data chronologically per participant
  arrange(participant_ID, Timestamp) %>%
  group_by(participant_ID) %>%
  mutate(
    # Make sure Timestamp is numeric (sometimes CSVs read them as text)
    Timestamp_num = as.numeric(Timestamp),

    # 2. Calculate the gap between the current row and the previous row
    # (default = first(Timestamp_num) prevents NA on the very first row)
    time_diff_ms = Timestamp_num - lag(Timestamp_num, default = first(Timestamp_num)),

    # 3. Check if the gap is greater than 300,000 ms (5 minutes)
    is_new_block = time_diff_ms > 60000,

    # 4. cumsum() counts how many times the 5-min gap happened.
    # Adding 1 ensures the first chunk starts at Block 1 instead of 0.
    Block_Number = cumsum(is_new_block) + 1,

    # Your existing formatting logic
    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
    prev_IR = lag(Accuracy.IR)
  ) %>%
  ungroup() %>%
  # Filter to just your target trials AFTER assigning the blocks
  filter(isTarget == TRUE)

# 2. Aggregate scores per Participant AND Block
blocked_aggregated <- time_blocked_data %>%
  group_by(participant_ID, Block_Number) %>%
  summarise(
    # IR components
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown", na.rm = TRUE),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy.IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    # IR metrics
    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,
```

```
    # WR components
    WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_total   = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

    # WR metric
    WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),
    .groups = "drop"
  ) %>%
  # Ensure the Block_Number is a categorical factor for plotting/testing
  # (If some participants had more than 3 blocks due to pausing, you can filter them here)
  filter(Block_Number %in% c(1, 2, 3)) %>%
  mutate(Block_Number = factor(Block_Number, levels = c(1, 2, 3), labels = c("Block 1", "Block 2", "Blo
```

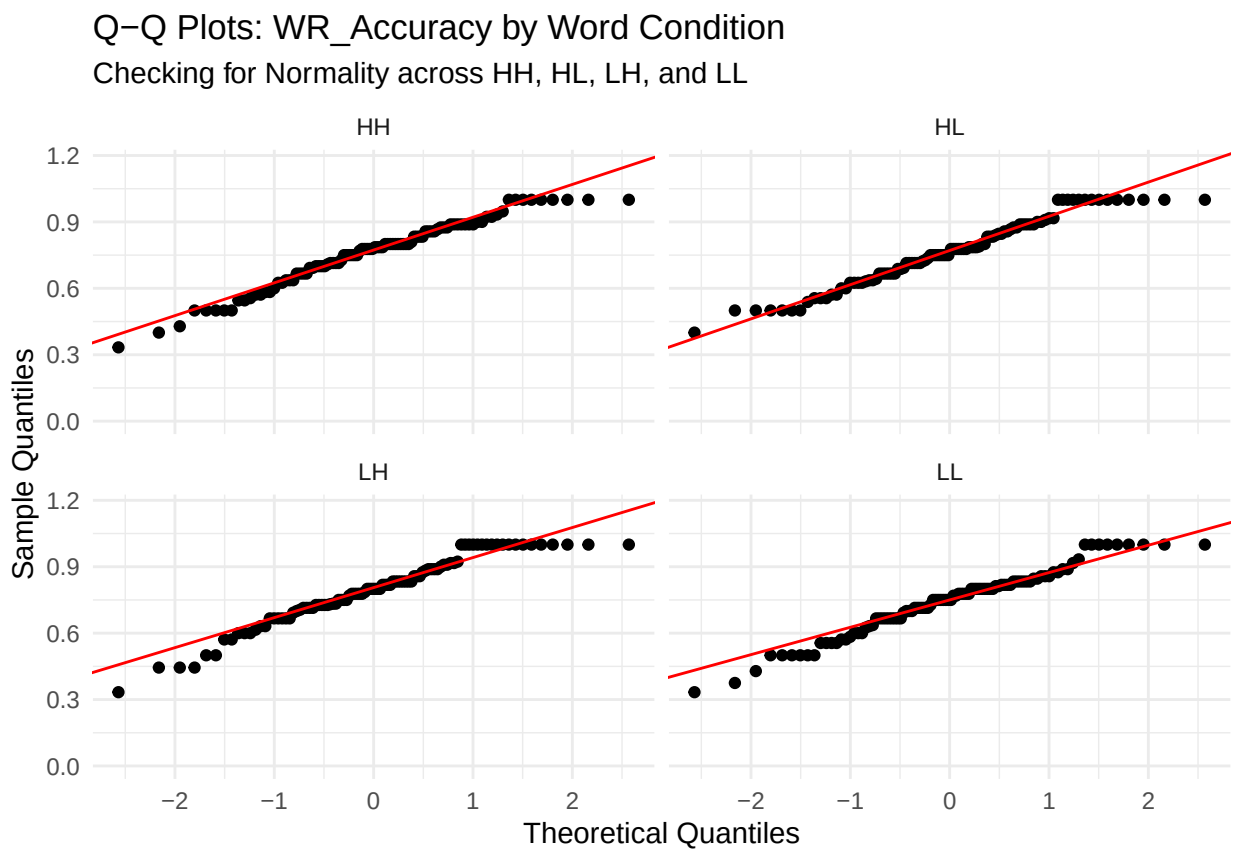
```
# Testing for Normality
# QQ Plots
library(ggplot2)

ggplot(aggregated_matrix, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Word_Condition, ncol = 2) + # This creates the 4-plot grid
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: IR Memorability by Word Condition",
       subtitle = "Checking for Normality across HH, HL, LH, and LL",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```



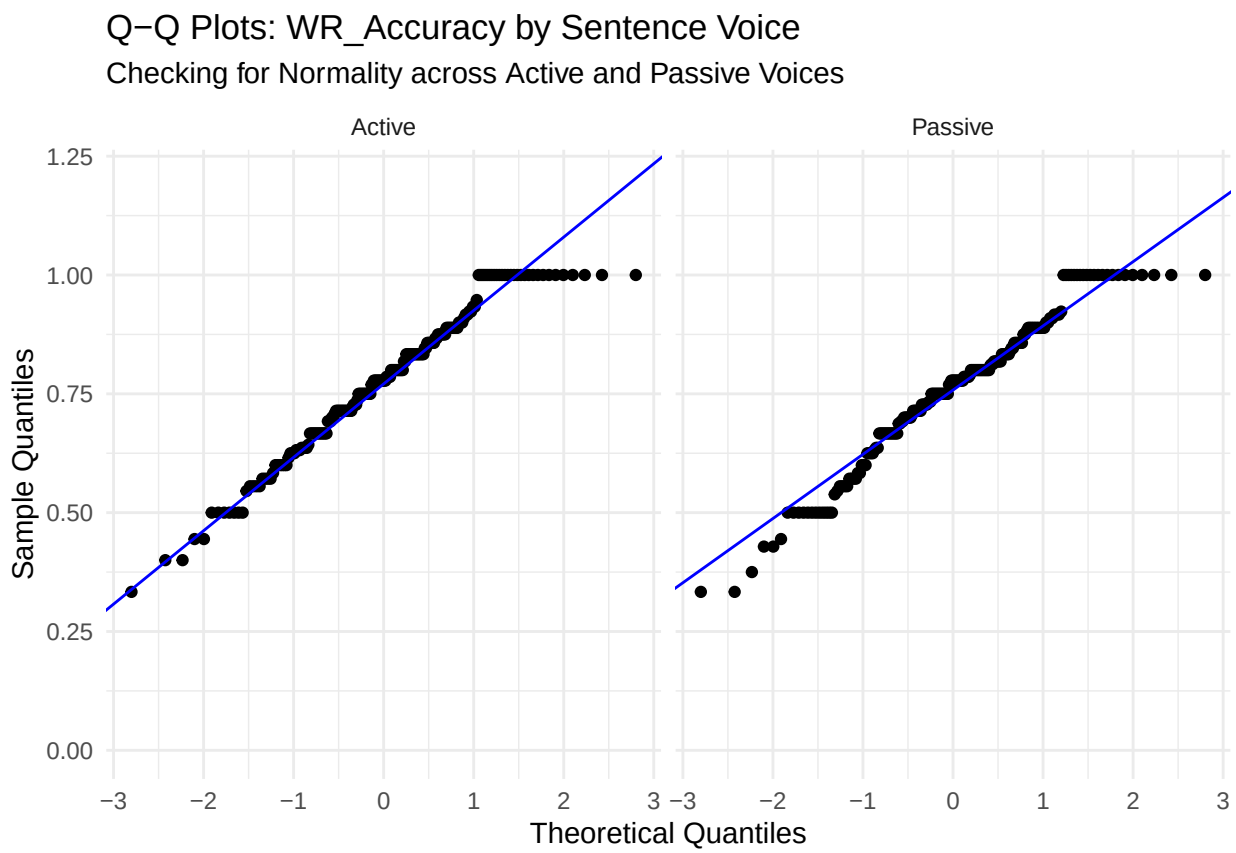

```r
ggplot(aggregated_matrix, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "blue") + # Using blue to distinguish from the Word Condition plots
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: IR Memorability by Sentence Voice",
       subtitle = "Checking for Normality across Active and Passive Voices",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```



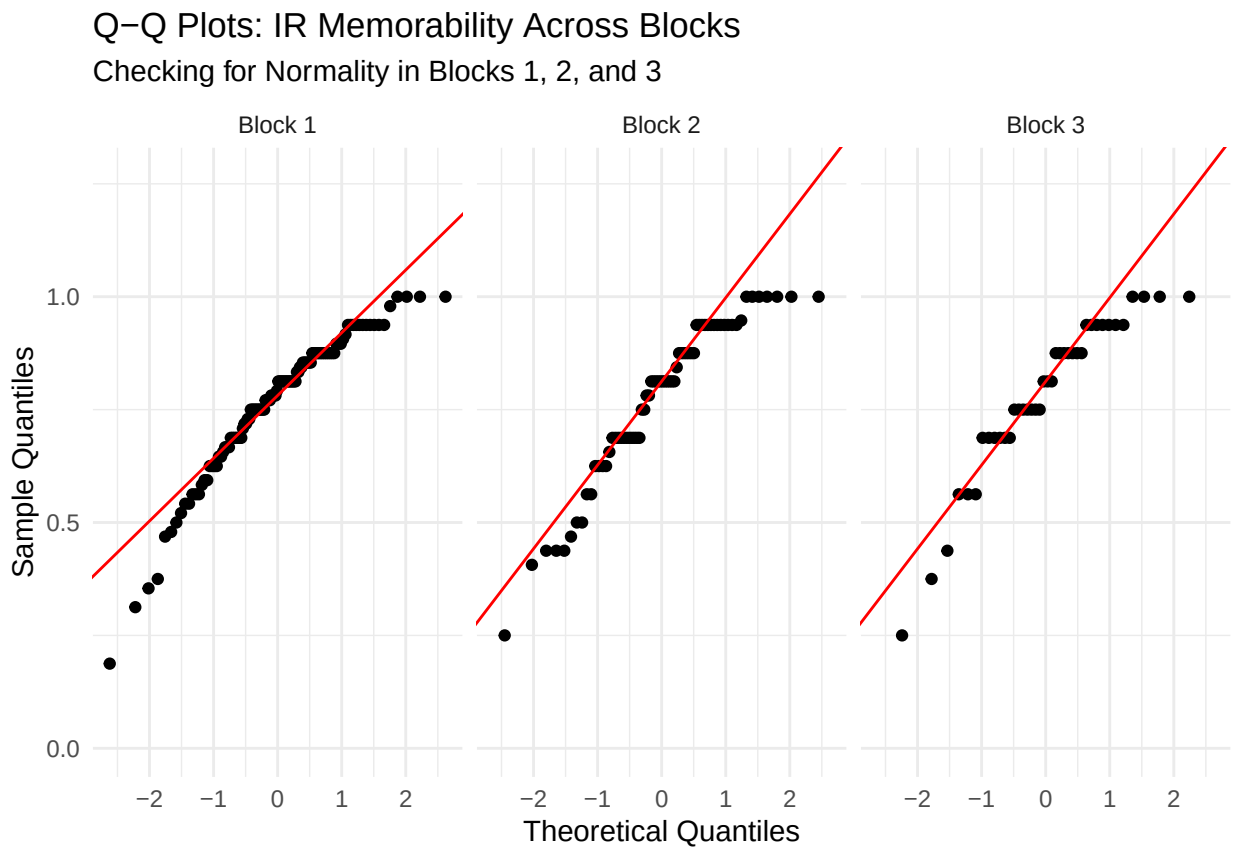

```r
ggplot(aggregated_matrix, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Word_Condition, ncol = 2) + # This creates the 4-plot grid
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: WR_Accuracy by Word Condition",
       subtitle = "Checking for Normality across HH, HL, LH, and LL",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```



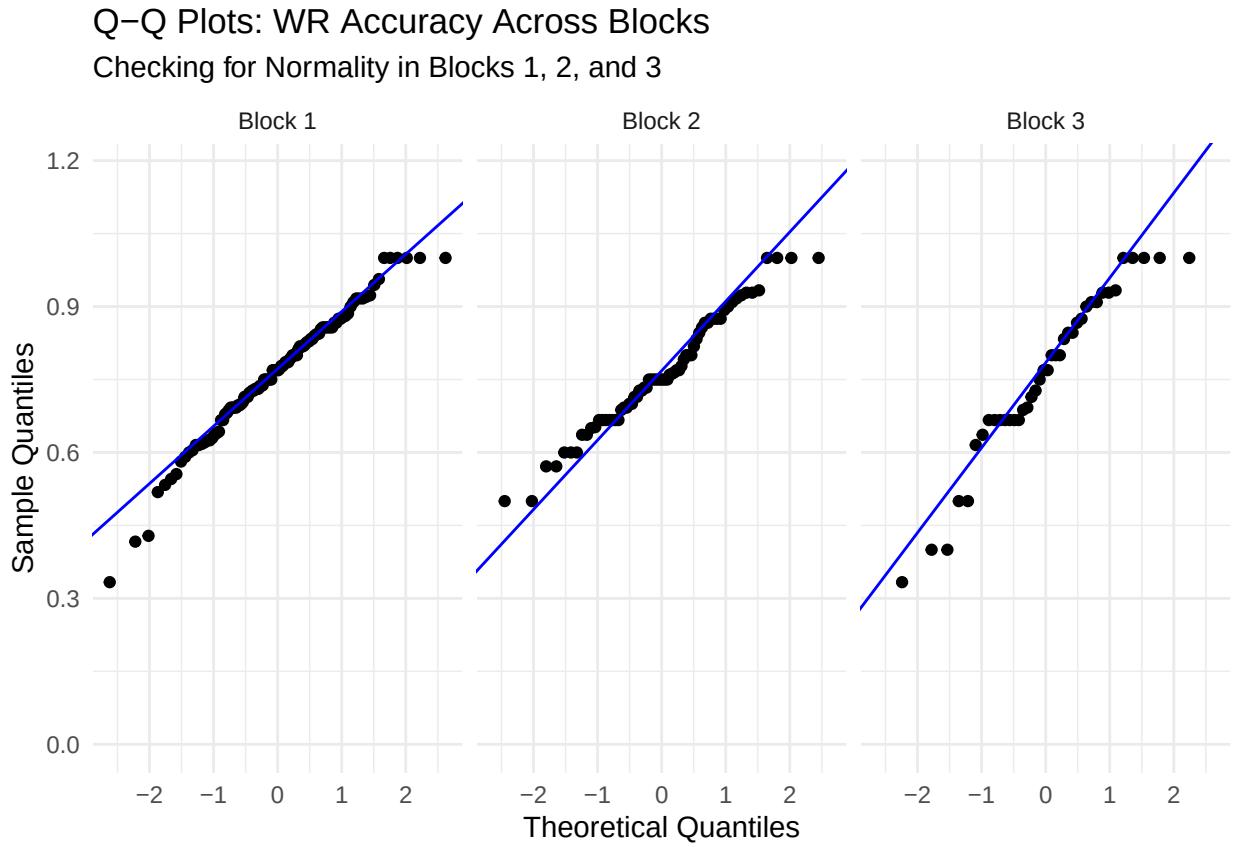

```r
ggplot(aggregated_matrix, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") + # Using blue to distinguish from the Word Condition plots
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: WR_Accuracy by Sentence Voice",
       subtitle = "Checking for Normality across Active and Passive Voices",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```




```r
library(ggplot2)

# Q-Q Plot for IR Memorability by Block
ggplot(blocked_aggregated, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Block_Number, ncol = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: IR Memorability Across Blocks",
       subtitle = "Checking for Normality in Blocks 1, 2, and 3",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```




```
# Q-Q Plot for WR Accuracy by Block
ggplot(blocked_aggregated, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") +
  facet_wrap(~ Block_Number, ncol = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: WR Accuracy Across Blocks",
       subtitle = "Checking for Normality in Blocks 1, 2, and 3",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```




```
# Anderson Darling
library(nortest)

aggregated_matrix %>%
  group_by(Word_Condition) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )
```

```
## # A tibble: 4 x 3
##   Word_Condition AD_Stat p_value
##   <fct>            <dbl>   <dbl>
## 1 HH               0.281 0.633
## 2 HL               0.628 0.0988
## 3 LH               1.22  0.00344
## 4 LL               0.445 0.278
```

```
aggregated_matrix %>%
  group_by(Sentence_Voice) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )
```

```
## # A tibble: 2 x 3
##   Sentence_Voice AD_Stat p_value
```

```
##   <fct>            <dbl>   <dbl>
## 1 Active            1.00 0.0118
## 2 Passive           1.09 0.00719
```

```
aggregated_matrix %>%
  group_by(Word_Condition) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )
```

```
## # A tibble: 4 x 3
##   Word_Condition AD_Stat p_value
##   <fct>            <dbl>   <dbl>
## 1 HH               0.794 0.0382
## 2 HL               0.708 0.0627
## 3 LH               1.13  0.00549
## 4 LL               0.819 0.0333
```

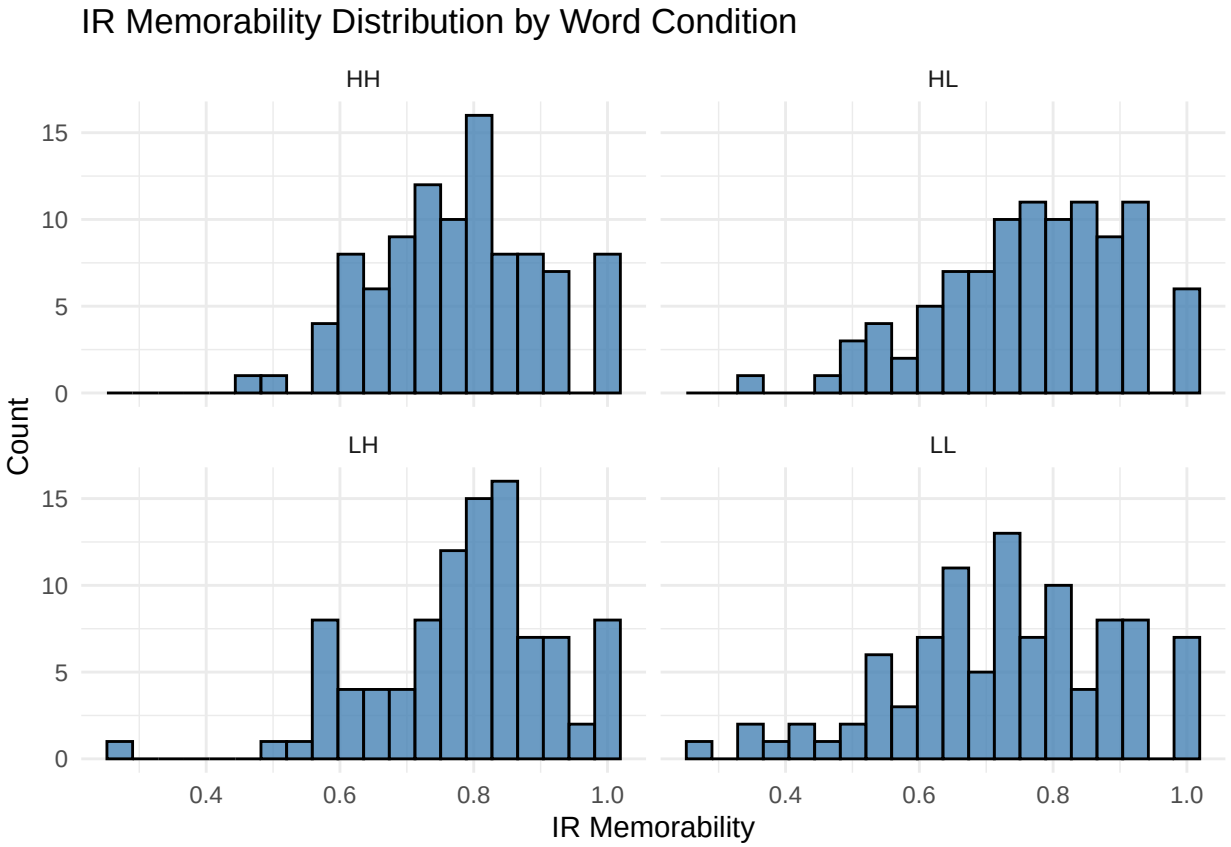
```
aggregated_matrix %>%
  group_by(Sentence_Voice) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )
```

```
## # A tibble: 2 x 3
##   Sentence_Voice AD_Stat  p_value
##   <fct>            <dbl>    <dbl>
## 1 Active            1.25 0.00297
## 2 Passive           1.54 0.000554
```

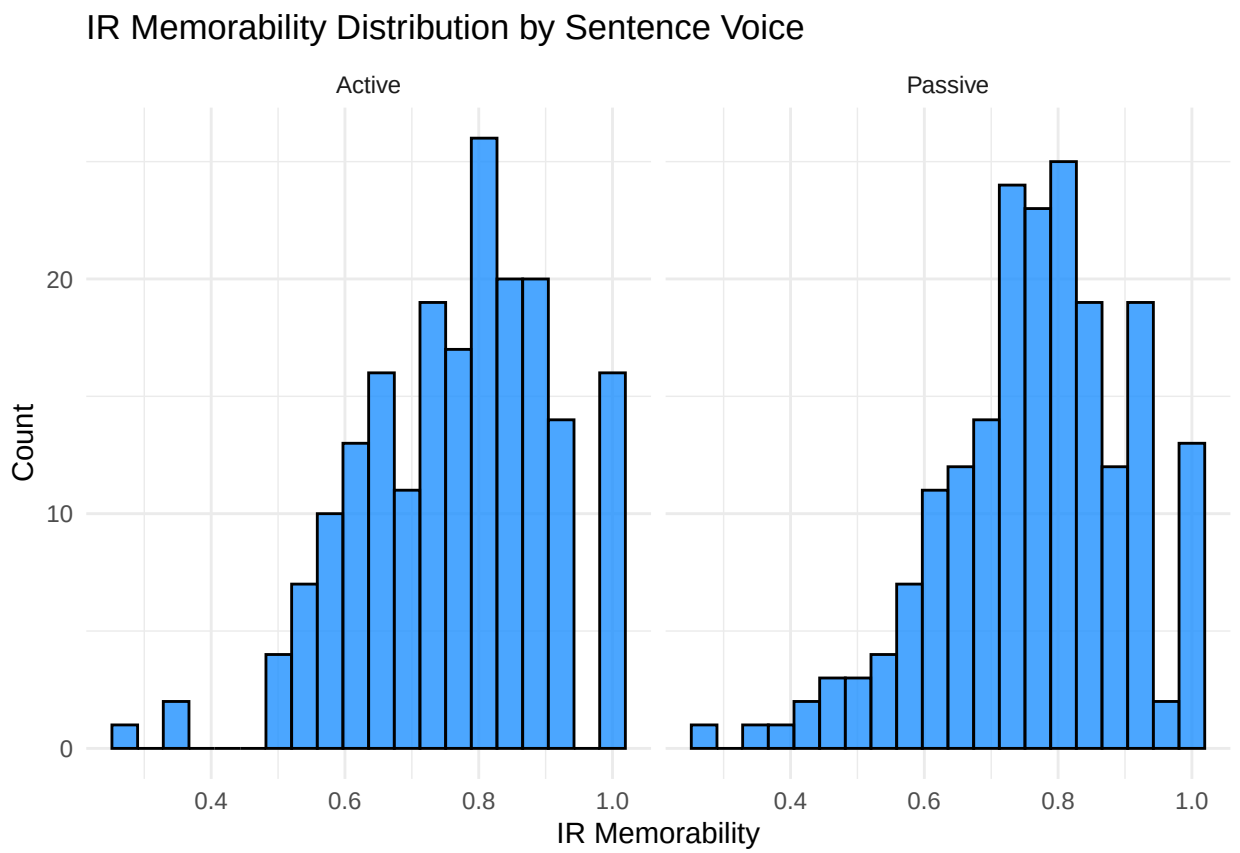
```
ir_normality_blocks <- blocked_aggregated %>%
  group_by(Block_Number) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )

wr_normality_blocks <- blocked_aggregated %>%
  group_by(Block_Number) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )
```

```
# Graph 1: Histogram of IR Memorability Scores
ggplot(aggregated_matrix, aes(x = IR_Memorability)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "black", alpha = 0.8) +
  facet_wrap(~ Word_Condition, ncol = 2) +
  labs(
    title = "IR Memorability Distribution by Word Condition",
    x = "IR Memorability",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
```

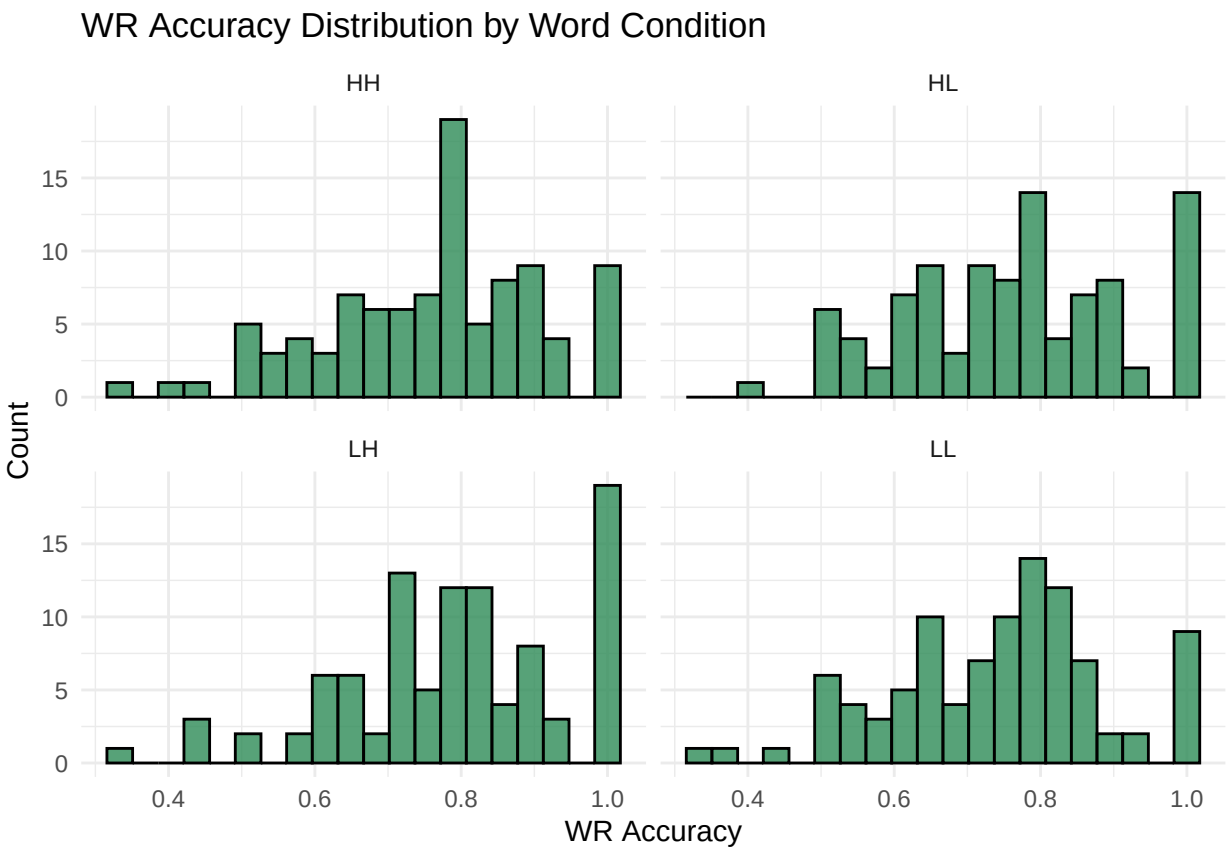
```
  theme_minimal()
```



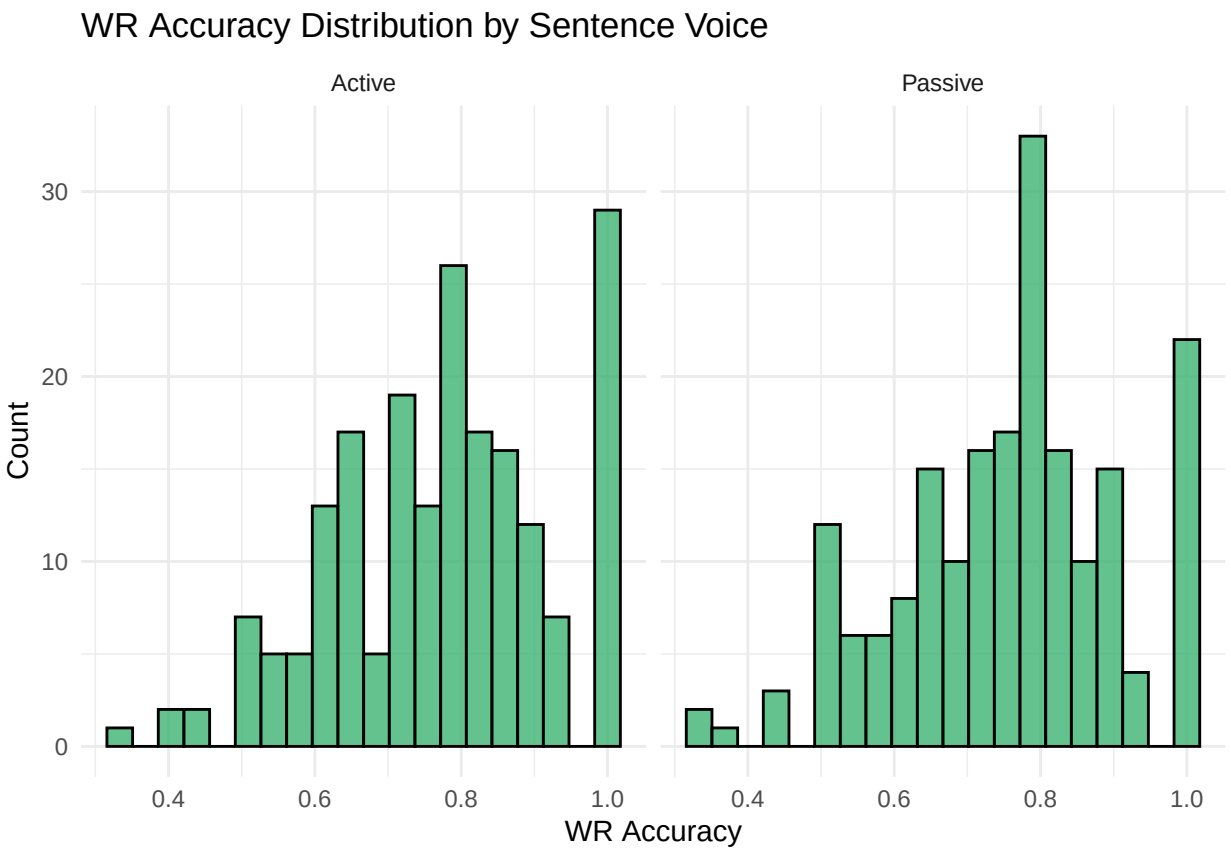

```
ggplot(aggregated_matrix, aes(x = IR_Memorability)) +
  geom_histogram(bins = 20, fill = "dodgerblue", color = "black", alpha = 0.8) +
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  labs(
    title = "IR Memorability Distribution by Sentence Voice",
    x = "IR Memorability",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```



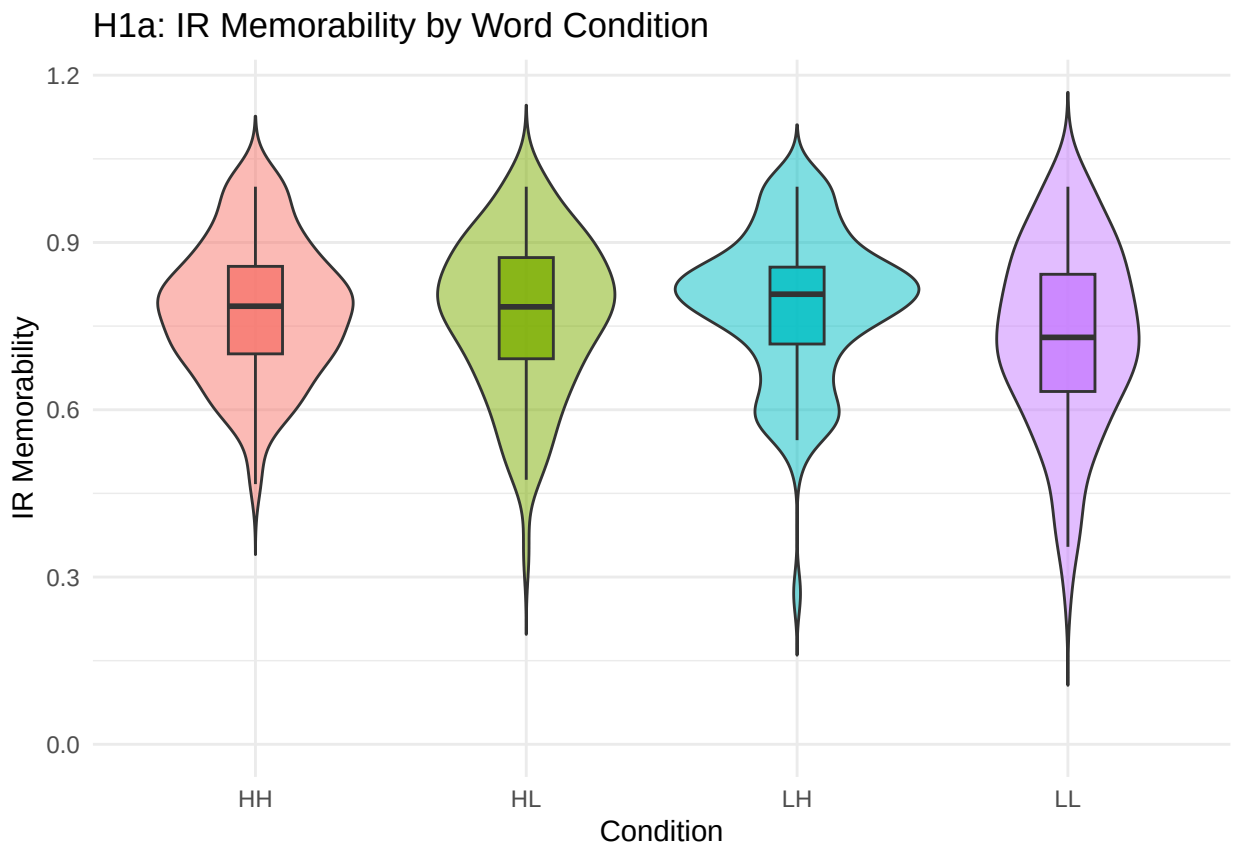

```r
# Graph 2: Histogram of WR Accuracy Scores
ggplot(aggregated_matrix, aes(x = WR_Accuracy)) +
  geom_histogram(bins = 20, fill = "seagreen", color = "black", alpha = 0.8) +
  facet_wrap(~ Word_Condition, ncol = 2) +
  labs(
    title = "WR Accuracy Distribution by Word Condition",
    x = "WR Accuracy",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```



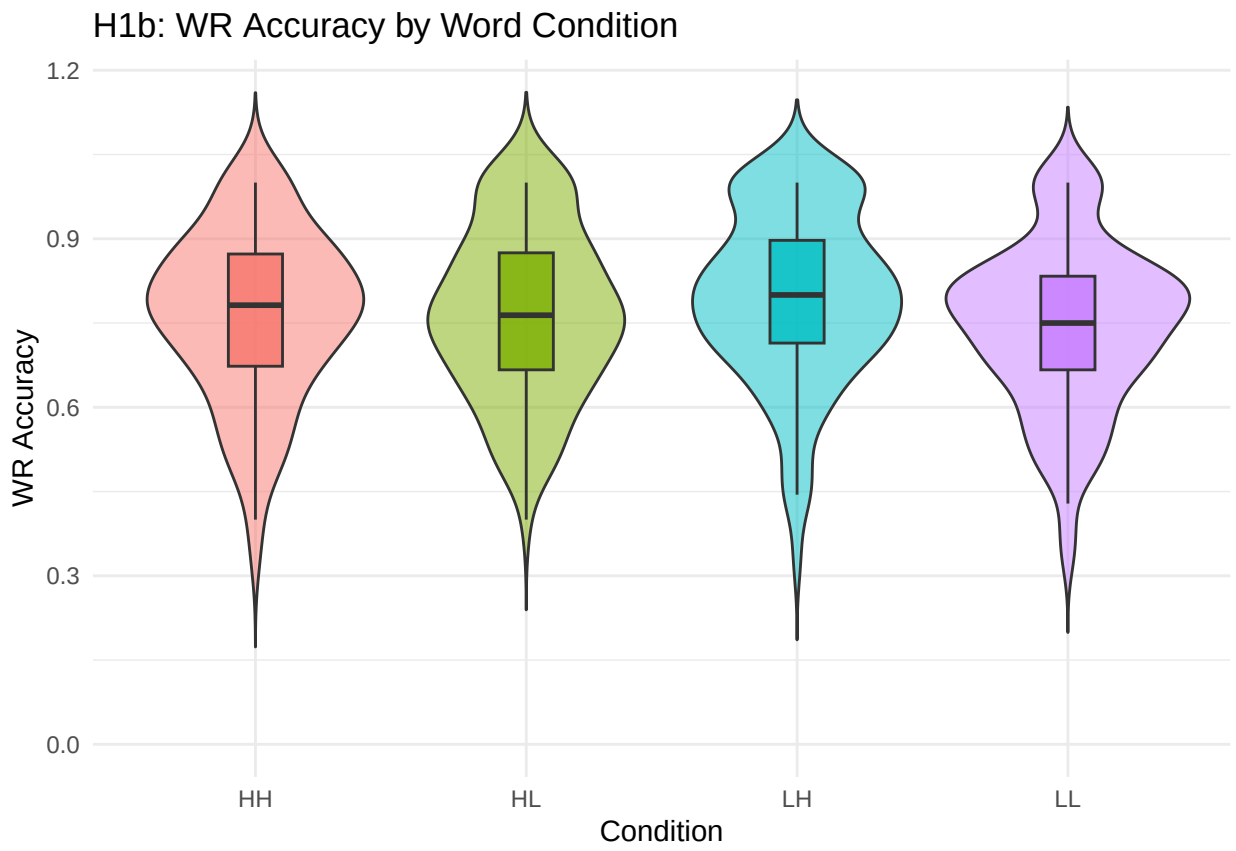

```
ggplot(aggregated_matrix, aes(x = WR_Accuracy)) +
  geom_histogram(bins = 20, fill = "mediumseagreen", color = "black", alpha = 0.8) +
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  labs(
    title = "WR Accuracy Distribution by Sentence Voice",
    x = "WR Accuracy",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```



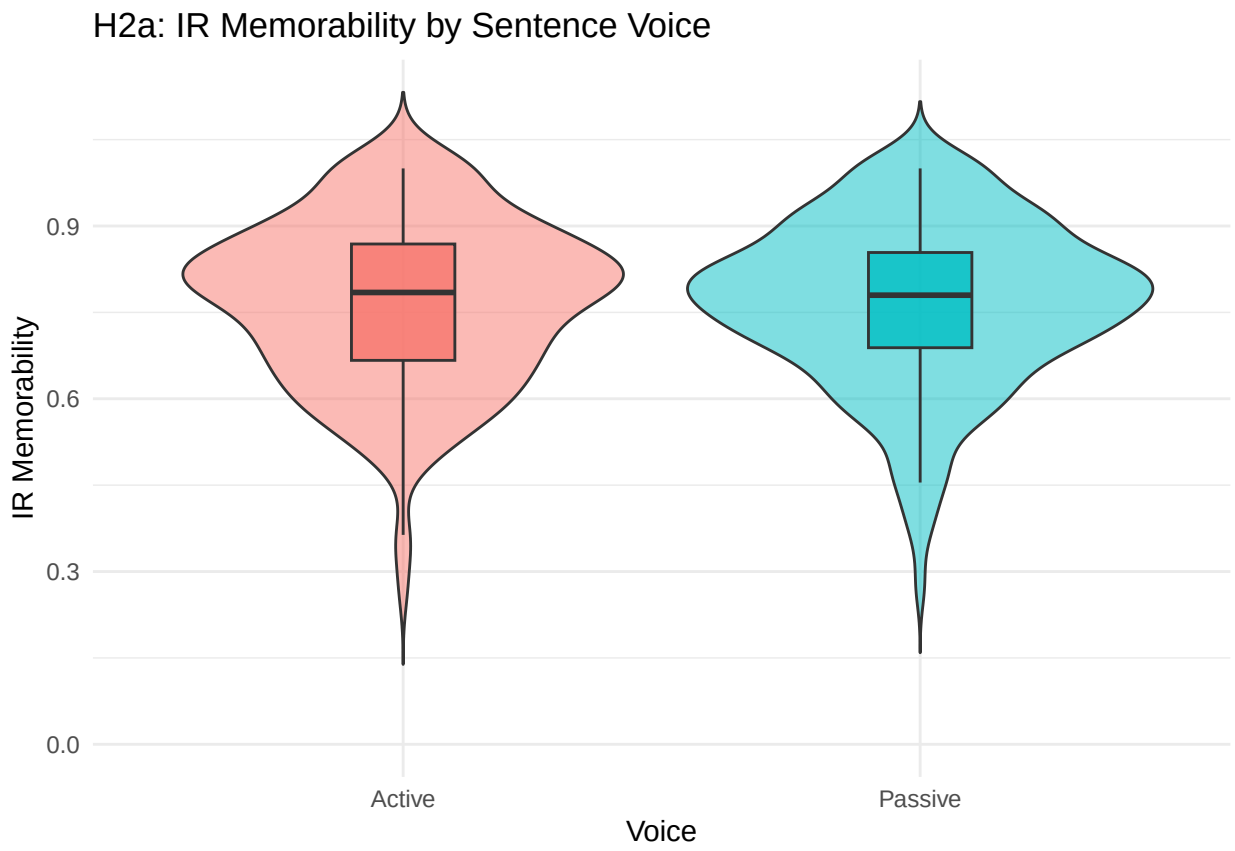

```
# Graph 3 (H1a): Violin & Boxplot for IR by Word Condition
ggplot(aggregated_matrix, aes(x = Word_Condition, y = IR_Memorability, fill = Word_Condition)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H1a: IR Memorability by Word Condition", x = "Condition", y = "IR Memorability") +
  theme_minimal() + theme(legend.position = "none")
```



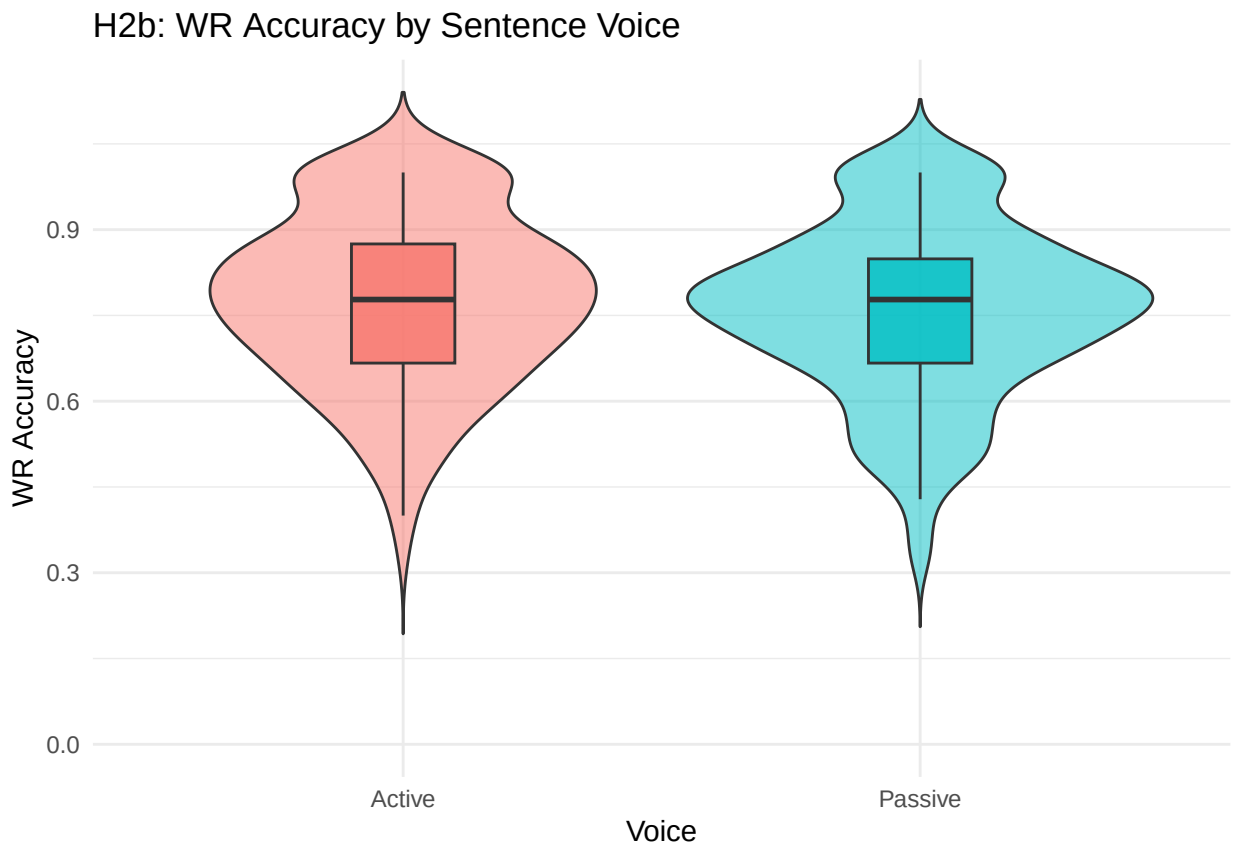

```
# Graph 4 (H1b): Violin & Boxplot for WR by Word Condition
ggplot(aggregated_matrix, aes(x = Word_Condition, y = WR_Accuracy, fill = Word_Condition)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H1b: WR Accuracy by Word Condition", x = "Condition", y = "WR Accuracy") +
  theme_minimal() + theme(legend.position = "none")
```



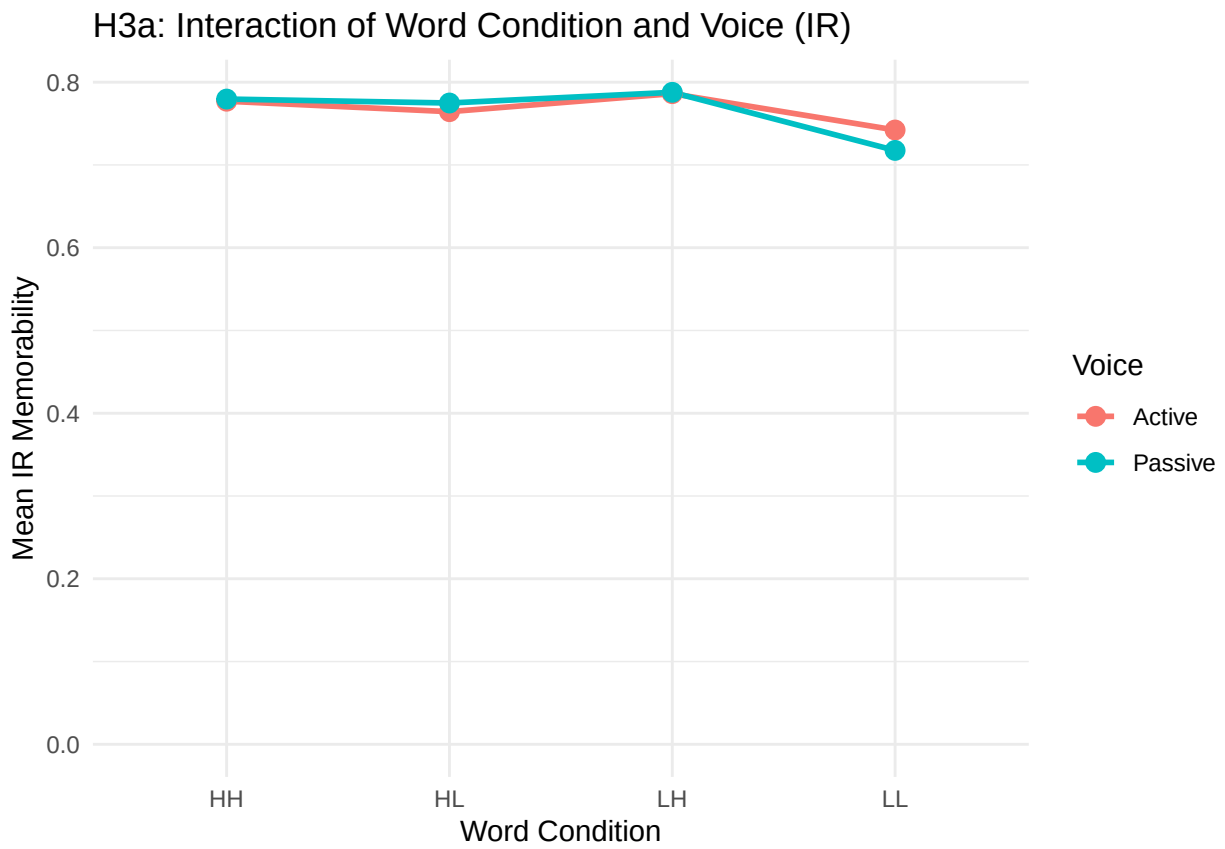

```
# Graph 5 (H2a): Violin & Boxplot for IR by Voice
ggplot(aggregated_matrix %>% drop_na(Sentence_Voice), aes(x = Sentence_Voice, y = IR_Memorability, fill
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H2a: IR Memorability by Sentence Voice", x = "Voice", y = "IR Memorability") +
  theme_minimal() + theme(legend.position = "none")
```



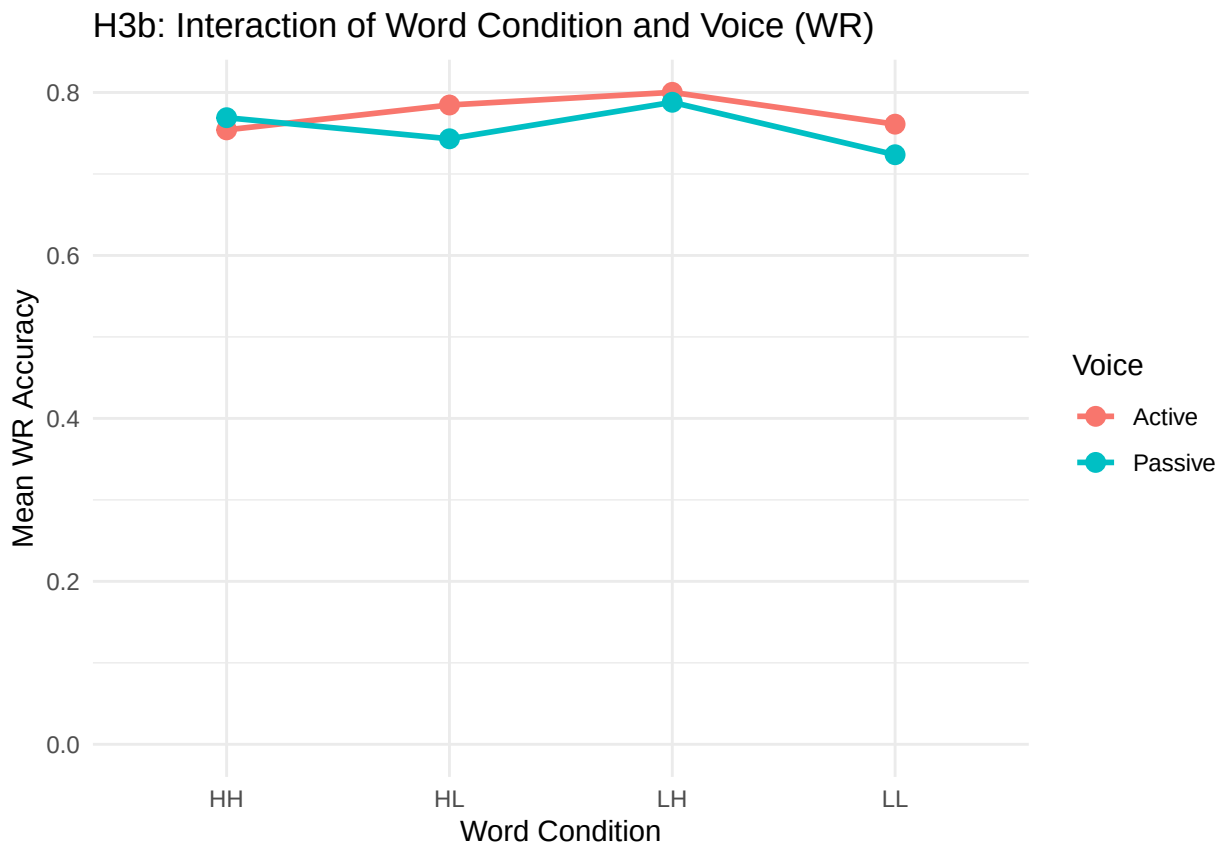

```
# Graph 6 (H2b): Violin & Boxplot for WR by Voice
ggplot(aggregated_matrix %>% drop_na(Sentence_Voice), aes(x = Sentence_Voice, y = WR_Accuracy, fill = S
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H2b: WR Accuracy by Sentence Voice", x = "Voice", y = "WR Accuracy") +
  theme_minimal() + theme(legend.position = "none")
```



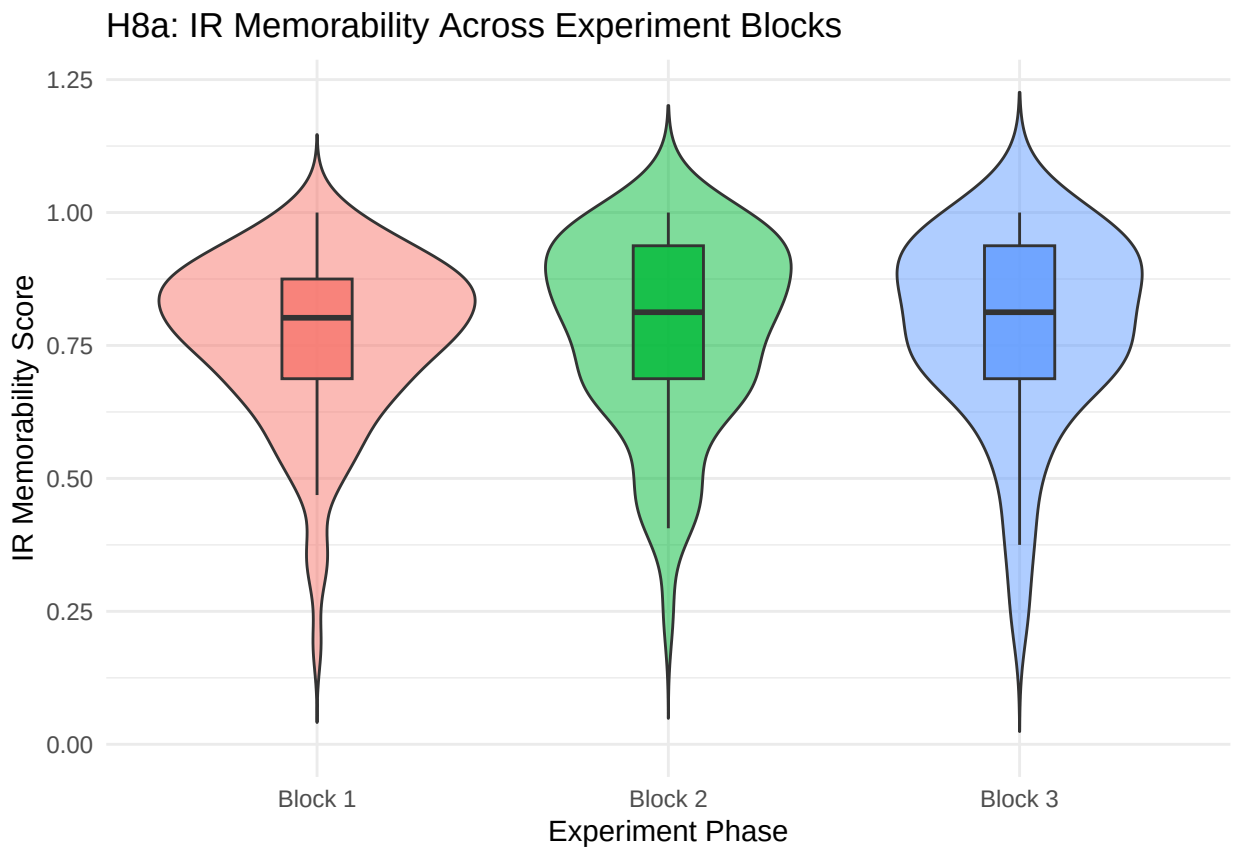

```
interaction_data <- aggregated_matrix %>%
  drop_na(Word_Condition, Sentence_Voice) %>%
  group_by(Word_Condition, Sentence_Voice) %>%
  summarise(
    Mean_IR = mean(IR_Memorability, na.rm = TRUE),
    Mean_WR = mean(WR_Accuracy, na.rm = TRUE),
    .groups = 'drop'
  )

# Graph 7 (H3a): Interaction Plot for IR
ggplot(interaction_data, aes(x = Word_Condition, y = Mean_IR, color = Sentence_Voice, group = Sentence_
  geom_line(linewidth = 1) + geom_point(size = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H3a: Interaction of Word Condition and Voice (IR)", x = "Word Condition", y = "Mean IR
  theme_minimal()
```




```
# Graph 8 (H3b): Interaction Plot for WR
ggplot(interaction_data, aes(x = Word_Condition, y = Mean_WR, color = Sentence_Voice, group = Sentence_V
  geom_line(linewidth = 1) + geom_point(size = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H3b: Interaction of Word Condition and Voice (WR)", x = "Word Condition", y = "Mean WR A
  theme_minimal()
```

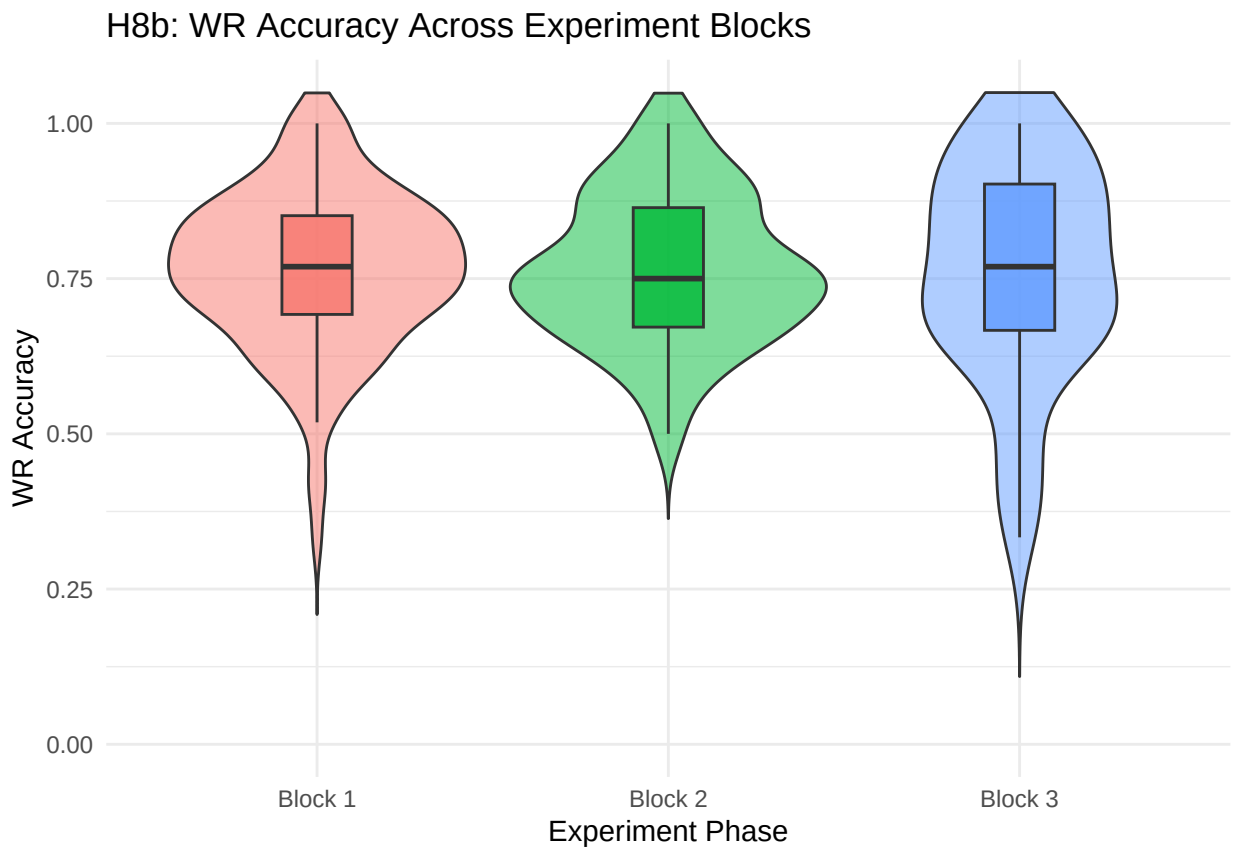



```
# Graph H8a: Violin & Boxplot for IR Memorability across Blocks
ggplot(blocked_aggregated, aes(x = Block_Number, y = IR_Memorability, fill = Block_Number)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H8a: IR Memorability Across Experiment Blocks",
       x = "Experiment Phase",
       y = "IR Memorability Score") +
  theme_minimal() +
  theme(legend.position = "none")
```




```
# Graph H8b: Violin & Boxplot for WR Accuracy across Blocks
ggplot(blocked_aggregated, aes(x = Block_Number, y = WR_Accuracy, fill = Block_Number)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, 1.05)) +  # Cap at 1.05 so the 1.0 ceiling is visible
  labs(title = "H8b: WR Accuracy Across Experiment Blocks",
       x = "Experiment Phase",
       y = "WR Accuracy") +
  theme_minimal() +
  theme(legend.position = "none")
```

```
## Warning: Removed 180 rows containing missing values or values outside the scale range
## (`geom_violin()`).
```




```
# Kruskal-Wallis Test for IR Memorability
kw_ir_block <- kruskal.test(IR_Memorability ~ Block_Number, data = blocked_aggregated)
print(kw_ir_block)
```

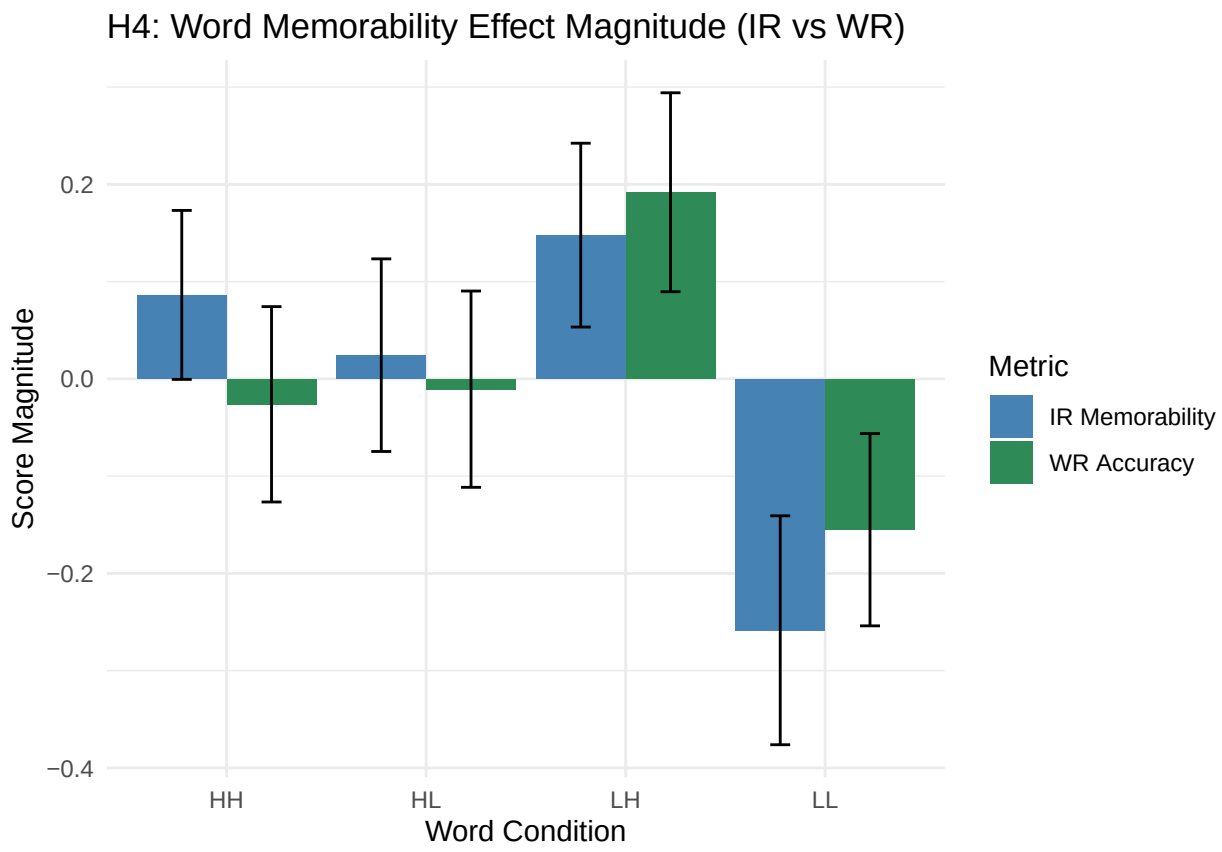
```
##
##  Kruskal-Wallis rank sum test
##
## data:  IR_Memorability by Block_Number
## Kruskal-Wallis chi-squared = 1.1188, df = 2, p-value = 0.5715
```

```
# Kruskal-Wallis Test for WR Accuracy
kw_wr_block <- kruskal.test(WR_Accuracy ~ Block_Number, data = blocked_aggregated)
print(kw_wr_block)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  WR_Accuracy by Block_Number
## Kruskal-Wallis chi-squared = 0.1085, df = 2, p-value = 0.9472
```

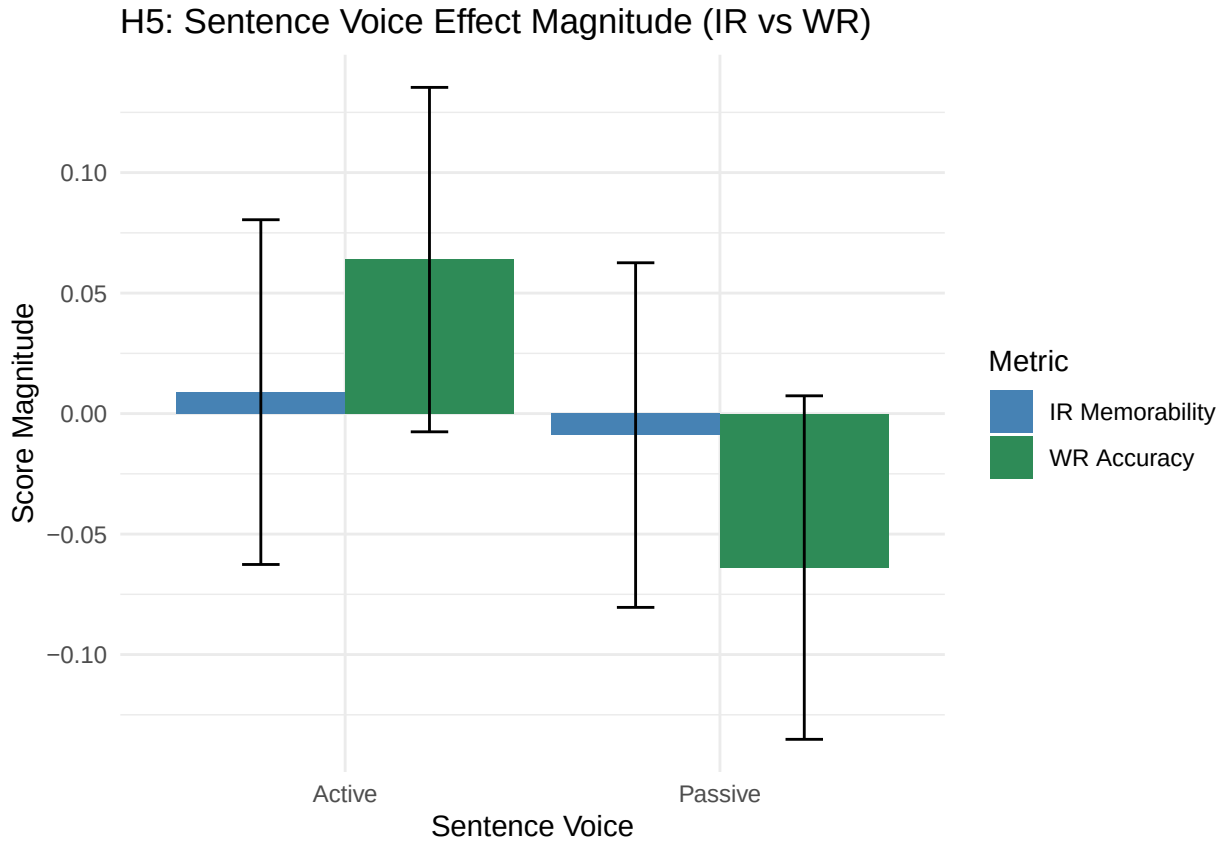
```
aggregated_scaled <- aggregated_matrix %>%
  mutate(
    IR_scaled = as.numeric(scale(IR_Memorability)),
    WR_scaled = as.numeric(scale(WR_Accuracy))
  )

# Calculate average scaled IR and WR scores and their standard errors for each word condition
effect_data <- aggregated_scaled %>%
```

```
  drop_na(Word_Condition, Sentence_Voice) %>%
  group_by(Word_Condition) %>%
  summarise(
    IR_mean = mean(IR_scaled, na.rm=TRUE),
    IR_se = sd(IR_scaled, na.rm=TRUE)/sqrt(n()),
    WR_mean = mean(WR_scaled, na.rm=TRUE),
    WR_se = sd(WR_scaled, na.rm=TRUE)/sqrt(n()),
    .groups="drop"
  )

# Graph 9 (H4): Word Memorability effects on IR vs WR
effect_data_long <- effect_data %>%
  pivot_longer(cols = c(IR_mean, WR_mean), names_to = "Metric", values_to = "Mean") %>%
  mutate(SE = ifelse(Metric == "IR_mean", IR_se, WR_se))

ggplot(effect_data_long, aes(x = Word_Condition, y = Mean, fill = Metric)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE), width = 0.2, position = position_dodge(0.9)) +
  labs(title = "H4: Word Memorability Effect Magnitude (IR vs WR)", x = "Word Condition", y = "Score Mag
  scale_fill_manual(values = c("steelblue", "seagreen"), labels = c("IR Memorability", "WR Accuracy")) +
  theme_minimal()
```




```
# Graph 10 (H5): Sentence Voice effects on IR vs WR
voice_effect <- aggregated_scaled %>%
  drop_na(Sentence_Voice) %>%
  group_by(Sentence_Voice) %>%
```

```r
  summarise(
    IR_mean = mean(IR_scaled, na.rm=TRUE),
    IR_se = sd(IR_scaled, na.rm=TRUE)/sqrt(n()),
    WR_mean = mean(WR_scaled, na.rm=TRUE),
    WR_se = sd(WR_scaled, na.rm=TRUE)/sqrt(n()),
    .groups="drop"
  ) %>%
  pivot_longer(cols = c(IR_mean, WR_mean), names_to = "Metric", values_to = "Mean") %>%
  mutate(SE = ifelse(Metric == "IR_mean", IR_se, WR_se))

ggplot(voice_effect, aes(x = Sentence_Voice, y = Mean, fill = Metric)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE), width = 0.2, position = position_dodge(0.9)) +
  labs(title = "H5: Sentence Voice Effect Magnitude (IR vs WR)", x = "Sentence Voice", y = "Score Magni
  scale_fill_manual(values = c("steelblue", "seagreen"), labels = c("IR Memorability", "WR Accuracy")) +
  theme_minimal()
```




```r
# Descriptive stats by Word Condition
desc_word <- aggregated_matrix %>%
  drop_na(Word_Condition) %>%
  group_by(Word_Condition) %>%
  summarise(
    IR_mean = mean(IR_Memorability, na.rm = TRUE),
    IR_sd = sd(IR_Memorability, na.rm = TRUE),
    IR_median = median(IR_Memorability, na.rm = TRUE),
```

```
    WR_mean = mean(WR_Accuracy, na.rm = TRUE),
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),
    WR_median = median(WR_Accuracy, na.rm = TRUE),

    n = n(),
    .groups = "drop"
  )

print(desc_word)
```

```
## # A tibble: 4 x 8
##   Word_Condition IR_mean IR_sd IR_median WR_mean WR_sd WR_median     n
##   <fct>            <dbl> <dbl>     <dbl>   <dbl> <dbl>     <dbl> <int>
## 1 HH               0.778 0.121     0.786   0.762 0.148     0.782    98
## 2 HL               0.770 0.138     0.785   0.764 0.149     0.764    98
## 3 LH               0.787 0.131     0.807   0.794 0.151     0.8      98
## 4 LL               0.730 0.164     0.730   0.742 0.146     0.75     98
```

```
# Descriptive stats by Sentence Voice
desc_voice <- aggregated_matrix %>%
  drop_na(Sentence_Voice) %>%
  group_by(Sentence_Voice) %>%
  summarise(
    IR_mean = mean(IR_Memorability, na.rm = TRUE),
    IR_sd = sd(IR_Memorability, na.rm = TRUE),
    IR_median = median(IR_Memorability, na.rm = TRUE),

    WR_mean = mean(WR_Accuracy, na.rm = TRUE),
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),
    WR_median = median(WR_Accuracy, na.rm = TRUE),

    n = n(),
    .groups = "drop"
  )

print(desc_voice)
```

```
## # A tibble: 2 x 8
##   Sentence_Voice IR_mean IR_sd IR_median WR_mean WR_sd WR_median     n
##   <fct>            <dbl> <dbl>     <dbl>   <dbl> <dbl>     <dbl> <int>
## 1 Active           0.767 0.141     0.785   0.775 0.149     0.778   196
## 2 Passive          0.765 0.141     0.780   0.756 0.149     0.778   196
```

```
kruskal_ir_word <- kruskal.test(IR_Memorability ~ Word_Condition, data = aggregated_matrix)
print(kruskal_ir_word)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  IR_Memorability by Word_Condition
## Kruskal-Wallis chi-squared = 7.3923, df = 3, p-value = 0.06039
```

```
kruskal_wr_word <- kruskal.test(WR_Accuracy ~ Word_Condition, data = aggregated_matrix)
print(kruskal_wr_word)
```

```
##
```

```
##  Kruskal-Wallis rank sum test
##
## data:  WR_Accuracy by Word_Condition
## Kruskal-Wallis chi-squared = 6.194, df = 3, p-value = 0.1025
```

```
mw_ir_voice <- wilcox.test(
  IR_Memorability ~ Sentence_Voice,
  data = aggregated_matrix,
  exact = FALSE
)

print(mw_ir_voice)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  IR_Memorability by Sentence_Voice
## W = 19350, p-value = 0.8996
## alternative hypothesis: true location shift is not equal to 0
```

```
mw_wr_voice <- wilcox.test(
  WR_Accuracy ~ Sentence_Voice,
  data = aggregated_matrix,
  exact = FALSE
)

print(mw_wr_voice)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  WR_Accuracy by Sentence_Voice
## W = 20505, p-value = 0.2468
## alternative hypothesis: true location shift is not equal to 0
```

```
# Create categorical correctness variables
processed_trials <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  group_by(participant_ID, Stimulus) %>%
  mutate(
    Word_Condition = case_when(
      str_detect(Stimulus, "^HH_")  ~ "HH",
      str_detect(Stimulus, "^HVL_") ~ "HL",
      str_detect(Stimulus, "^LVH_") ~ "LH",
      str_detect(Stimulus, "^LVL_") ~ "LL"
    ),
    Word_Condition = factor(Word_Condition, levels = c("HH", "HL", "LH", "LL")),

    Sentence_Voice = case_when(
      str_detect(Stimulus, "_A$") ~ "Active",
      str_detect(Stimulus, "_P$") ~ "Passive",
      TRUE ~ NA_character_
    ),
    Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),
```

```
    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
    prev_IR = lag(Accuracy.IR)
  ) %>%
  ungroup()

ir_data <- processed_trials %>%
  filter(Event == "IR pressed") %>%
  mutate(
    IR_Result = case_when(
      isRepeat == TRUE & Accuracy.IR == 1 ~ "Correct",
      is.na(isRepeat) ~ "Incorrect",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(IR_Result))

wr_data <- processed_trials %>%
  filter(Event == "WR pressed", prev_IR == 1) %>%
  mutate(
    WR_Result = case_when(
      Accuracy.WR == 1 ~ "Correct",
      Accuracy.WR == 0 ~ "Incorrect",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(WR_Result))



ir_word_table <- table(ir_data$Word_Condition, ir_data$IR_Result)
fisher_ir_word <- fisher.test(ir_word_table)
print(fisher_ir_word)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  ir_word_table
## p-value = 0.002997
## alternative hypothesis: two.sided
```

```
wr_word_table <- table(wr_data$Word_Condition, wr_data$WR_Result)
fisher_wr_word <- fisher.test(wr_word_table)
print(fisher_wr_word)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  wr_word_table
## p-value = 0.325
## alternative hypothesis: two.sided
```

```
ir_voice_table <- table(ir_data$Sentence_Voice, ir_data$IR_Result)
fisher_ir_voice <- fisher.test(ir_voice_table)
print(fisher_ir_voice)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  ir_voice_table
## p-value = 1
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.7347999 1.3411942
## sample estimates:
## odds ratio
##  0.9927989
```

```
wr_voice_table <- table(wr_data$Sentence_Voice, wr_data$WR_Result)
fisher_wr_voice <- fisher.test(wr_voice_table)
print(fisher_wr_voice)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  wr_voice_table
## p-value = 0.3423
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.9257805 1.2542198
## sample estimates:
## odds ratio
##   1.077499
```

```
# Create categorical correctness variables

ir_word_table <- table(ir_data$Word_Condition, ir_data$IR_Result)
chisq_ir_word <- chisq.test(ir_word_table)
print(chisq_ir_word)
```

```
##
##  Pearson's Chi-squared test
##
## data:  ir_word_table
## X-squared = 14.112, df = 3, p-value = 0.002756
```

```
wr_word_table <- table(wr_data$Word_Condition, wr_data$WR_Result)
chisq_wr_word <- chisq.test(wr_word_table)
print(chisq_wr_word)
```

```
##
##  Pearson's Chi-squared test
##
## data:  wr_word_table
## X-squared = 3.4461, df = 3, p-value = 0.3278
```

```
ir_voice_table <- table(ir_data$Sentence_Voice, ir_data$IR_Result)
chisq_ir_voice <- chisq.test(ir_voice_table)
print(chisq_ir_voice)
```

```
##
##  Pearson's Chi-squared test with Yates' continuity correction
##
```

```
## data:  ir_voice_table
## X-squared = 5.1175e-28, df = 1, p-value = 1
```

```
wr_voice_table <- table(wr_data$Sentence_Voice, wr_data$WR_Result)
chisq_wr_voice <- chisq.test(wr_voice_table)
print(chisq_wr_voice)
```

```
##
##  Pearson's Chi-squared test with Yates' continuity correction
##
## data:  wr_voice_table
## X-squared = 0.89244, df = 1, p-value = 0.3448
```

```
ir_wr_counts <- raw_dataset %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  group_by(participant_ID) %>%
  mutate(
    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
    prev_IR = lag(Accuracy.IR)
  ) %>%
  ungroup() %>%
  summarise(
    IR_Total = sum(Event == "IR pressed", na.rm = TRUE),
    IR_Correct = sum(Event == "IR pressed" & Accuracy.IR == 1 & isRepeat==TRUE, na.rm = TRUE),
    IR_Incorrect = sum(Event == "IR pressed" & Accuracy.IR == 0, na.rm = TRUE),

    WR_Total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),
    WR_Correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_Incorrect = sum(Event == "WR pressed" & Accuracy.WR == 0 & prev_IR == 1, na.rm = TRUE)
  )

print(ir_wr_counts)
```

```
## # A tibble: 1 x 6
##   IR_Total IR_Correct IR_Incorrect WR_Total WR_Correct WR_Incorrect
##      <int>      <int>        <int>    <int>      <int>        <int>
## 1     4508       4317          191     3872       2968          904
```